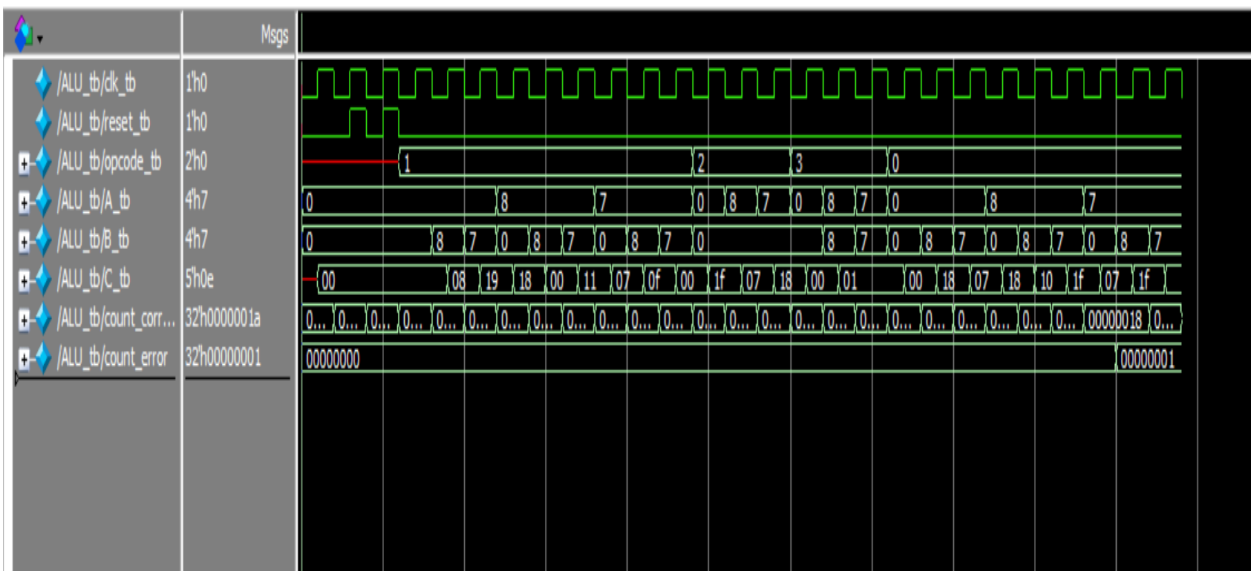
```
module ALU_4_bit (
    input  clk,
    input  reset,
    input  [1:0] Opcode,   // The opcode
    input  signed [3:0] A, // Input data A in 2's complement
    input  signed [3:0] B, // Input data B in 2's complement

    output reg signed [4:0] C // ALU output in 2's complement

        );

   reg signed [4:0]      Alu_out; // ALU output in 2's complement

   localparam         Add             = 2'b00; // A + B
   localparam         Sub             = 2'b01; // A - B
   localparam         Not_A            = 2'b10; // ~A
   localparam         ReductionOR_B  = 2'b11; // |B

   // Do the operation
   always @* begin
      case (Opcode)
         Add:            Alu_out = A + B;
         Sub:            Alu_out = A - B;
         Not_A:          Alu_out = ~A;
         ReductionOR_B:  Alu_out = |B;
        default:  Alu_out = 5'b0;
      endcase
   end // always @ *

   // Register output C
   always @(posedge clk or posedge reset) begin
      if (reset)
        C <= 5'b0;
      else
        C<= Alu_out;
   end

endmodule
```

```
module ALU_tb;

    // Declare local wire and reg
    reg clk_tb;
    reg reset_tb;
    reg [1:0] opcode_tb;
    reg signed [3:0] A_tb;
    reg signed [3:0] B_tb;
    wire signed [4:0] C_tb;

    // Instantiate the module under test
    ALU_4_bit uut(
        clk_tb,
        reset_tb,
        opcode_tb,
        A_tb,
        B_tb,
        C_tb
    );

    // define local parameter for tracking the functionality
    int  count_correct, count_error;

    // Define the boundaries for our No_systems(-2^(n-1) : -2^(n-1) - 1) ==> (-8--> 7)
    localparam MAXPOS = 7;
    localparam MAXNEG = -8;
    localparam  ZERO = 0;

    // define clock
    localparam T = 20;

    always begin
        clk_tb = 1'b0;
        #(T/2);
        clk_tb = 1'b1;
        #(T/2);
    end
```

```
// define task for checking
task checking(int expected_result);
    @(negedge clk_tb);
    if (expected_result !== C_tb) begin
        count_error += 1;
        $display("A = %d, B = %d, expected = %d, Actual = %d",A_tb,B_tb,expected_result,C_tb);
    end
    else begin
        count_correct += 1;
    end
endtask

// define task for reset functionality
task assert_reset;
    reset_tb = 1'b1;
    checking(0);
    reset_tb = 1'b0;
endtask

initial begin
    clk_tb = 1'b0;
    reset_tb = 1'b0;
    A_tb = 4'b0;
    B_tb = 4'b0;


// default case
    A_tb = ZERO; B_tb = ZERO;
    checking(ZERO);

    #10;
    assert_reset;
    #10;
    assert_reset;
```

```
////////////////////////////////////////////////////////////////
        // case1
        A_tb = ZERO; opcode_tb = 2'b01;
        B_tb = ZERO;
        checking(ZERO);
        B_tb = MAXNEG;
        checking(ZERO - MAXNEG);
        B_tb = MAXPOS;
        checking(ZERO - MAXPOS);

        // case2
        A_tb = MAXNEG; opcode_tb = 2'b01;
        B_tb = ZERO;
        checking(MAXNEG);
        B_tb = MAXNEG;
        checking(MAXNEG - MAXNEG);
        B_tb = MAXPOS;
        checking(MAXNEG - MAXPOS);

        // case3
        A_tb = MAXPOS; opcode_tb = 2'b01;
        B_tb = ZERO;
        checking(MAXPOS);
        B_tb = MAXNEG;
        checking(MAXPOS - MAXNEG);
        B_tb = MAXPOS;
        checking(MAXPOS - MAXPOS);

////////////////////////////////////////////////////////////////
        // case4
        A_tb = ZERO; opcode_tb = 2'b10;
        B_tb = ZERO;
        checking(~ZERO);

        // case5
        A_tb = MAXNEG; opcode_tb = 2'b10;
        B_tb = ZERO;
        checking(~MAXNEG);

        // case6
        A_tb = MAXPOS; opcode_tb = 2'b10;
        B_tb = ZERO;
        checking(~MAXPOS);
```

```
////////////////////////////////////////////////////////////////
        // case7
        A_tb = ZERO; opcode_tb = 2'b11;
        B_tb = ZERO;
        checking(|ZERO);


        // case8
        A_tb = MAXNEG; opcode_tb = 2'b11;
        B_tb = MAXNEG;
        checking(|MAXNEG);


        // case9
        A_tb = MAXPOS; opcode_tb = 2'b11;
        B_tb = MAXPOS;
        checking(|MAXPOS);
```



```
================================================================================
=== Instance: /ALU_tb/uut
=== Design Unit: work.ALU_4_bit
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         7         7         0   100.00%
```

```
 Statement Coverage:
     Enabled Coverage              Bins      Hits    Misses  Coverage
     ----------------              ----      ----    ------  --------
     Statements                       9         9         0   100.00%
```

```
 Toggle Coverage:
     Enabled Coverage              Bins      Hits    Misses  Coverage
     ----------------              ----      ----    ------  --------
     Toggles                         44        44         0   100.00%

 ================================Toggle Details================================

 Toggle Coverage for instance /ALU_tb/uut --

                                          Node      1H->0L      0L->1H  "Coverage"
                                          ----------------------------------------
                                         A[0-3]           1           1      100.00
                                     Alu_out[4-0]         1           1      100.00
                                         B[0-3]           1           1      100.00
                                         C[4-0]           1           1      100.00
                                      Opcode[0-1]         1           1      100.00
                                            clk           1           1      100.00
                                          reset           1           1      100.00

 Total Node Count     =         22
 Toggled Node Count   =         22
 Untoggled Node Count =          0
```